

Introduction to Algorithms

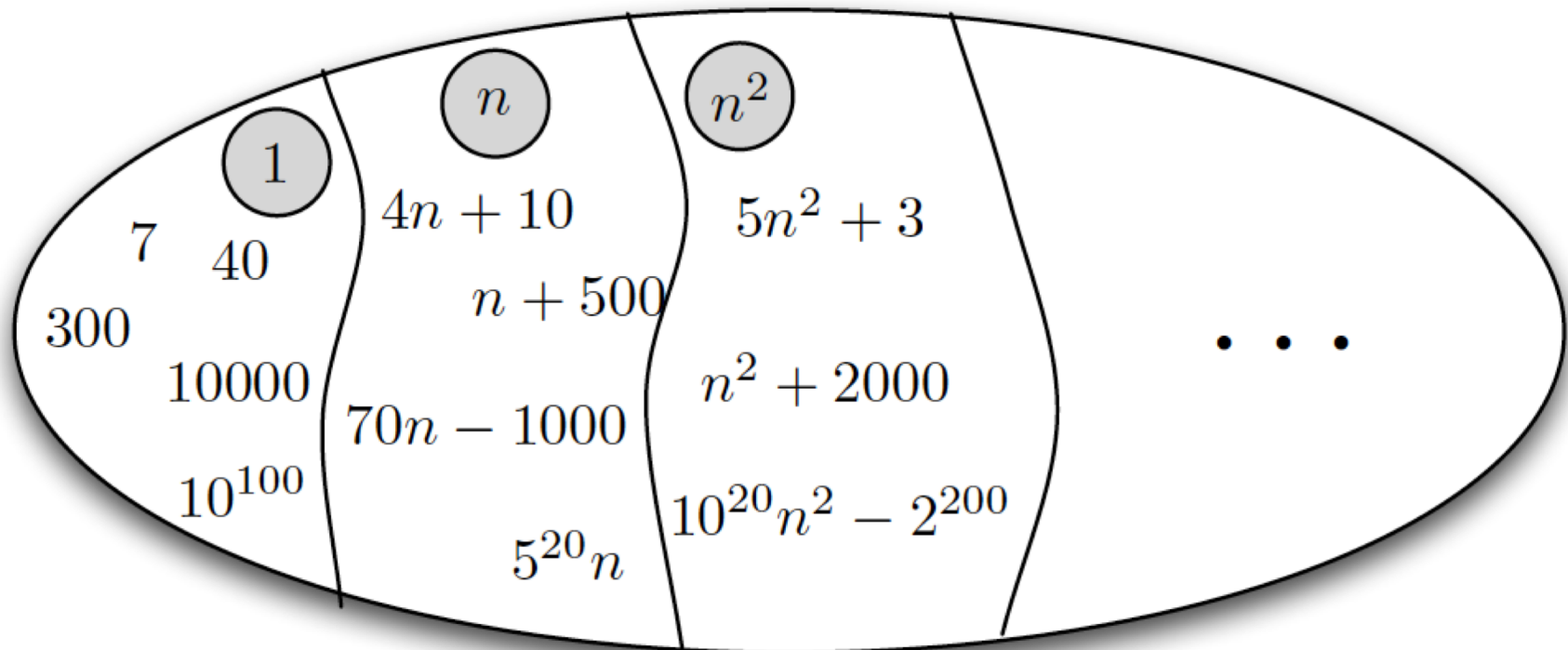
Email: grad.saeed@gmail.com

Github: github.com/saeedgrad

Asymptotic notation: formal definitions

- The notations we use to describe the asymptotic running time of an algorithm are defined in terms of functions whose domains are typically the set of natural numbers $\mathbb{N}$ or the set of real numbers $\mathbb{R}$.
- Such notations are convenient for describing a running time function $T(n)$.

Classifying the functions



Suppose that $f(n)$ and $g(n)$ are two functions, defined for non-negative integers n and with values in the set of real numbers.

Asymptotic upper bound on a function

***O*-notation**

As we saw in Section 3.1, *O*-notation describes an *asymptotic upper bound*. We use *O*-notation to give an upper bound on a function, to within a constant factor.

Here is the formal definition of *O*-notation. For a given function $g(n)$, we denote by $O(g(n))$ (pronounced “big-oh of g of n ” or sometimes just “oh of g of n ”) the *set of functions*

$$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that} \\ 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\} .^1$$

Asymptotic upper bound on a function

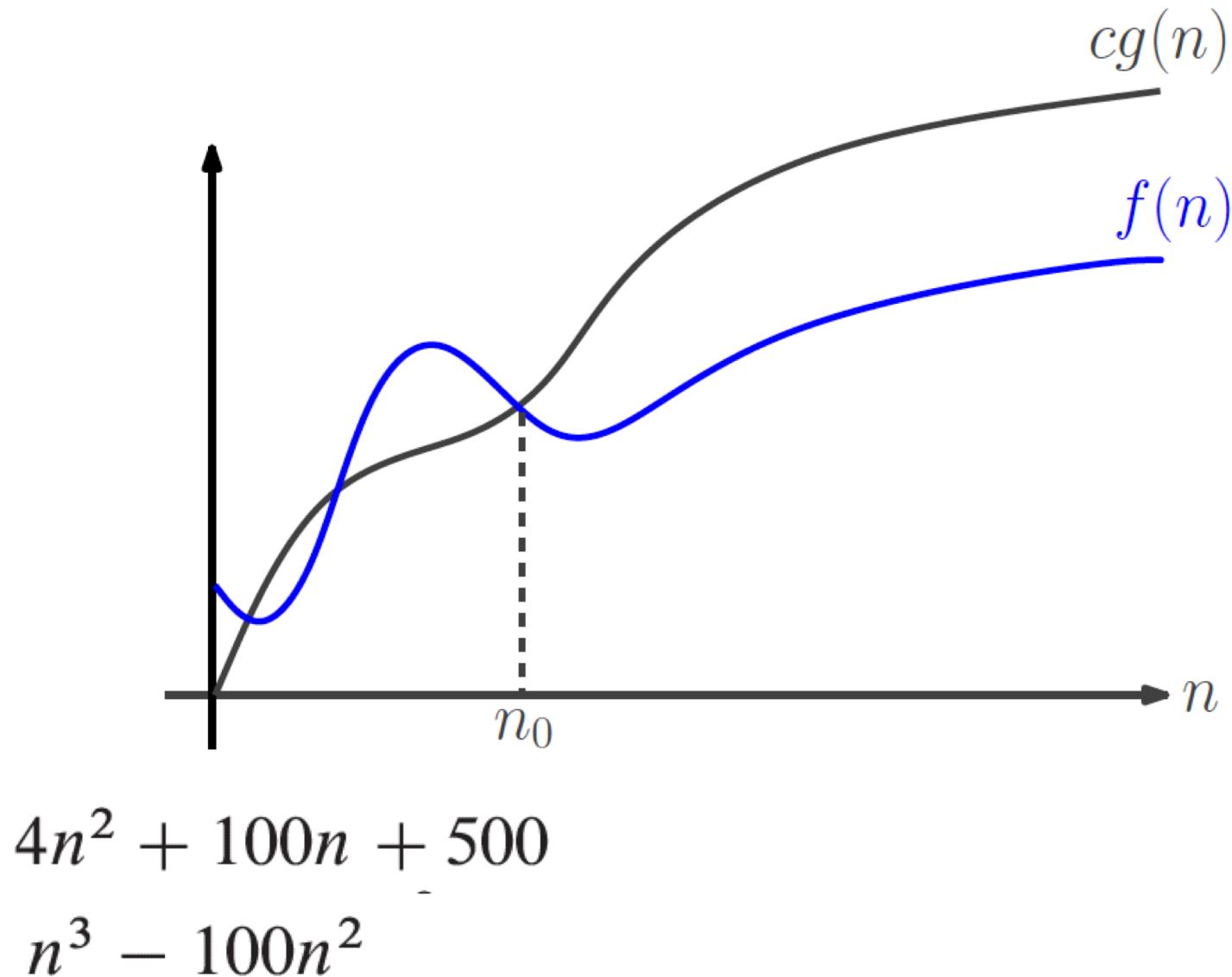
Big-O Notation

Definition

$f(n) = O(g(n))$ if and only if

$$\exists c > 0, \exists n_0 > 0 \ni \forall n \geq n_0, 0 < f(n) \leq c \cdot g(n).$$

Asymptotic upper bound on a function



Asymptotic lower bound on a function

Ω -notation

Just as O -notation provides an asymptotic *upper* bound on a function, Ω -notation provides an *asymptotic lower bound*. For a given function $g(n)$, we denote by $\Omega(g(n))$ (pronounced “big-omega of g of n ” or sometimes just “omega of g of n ”) the set of functions

$$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that} \\ 0 \leq c g(n) \leq f(n) \text{ for all } n \geq n_0\} .$$

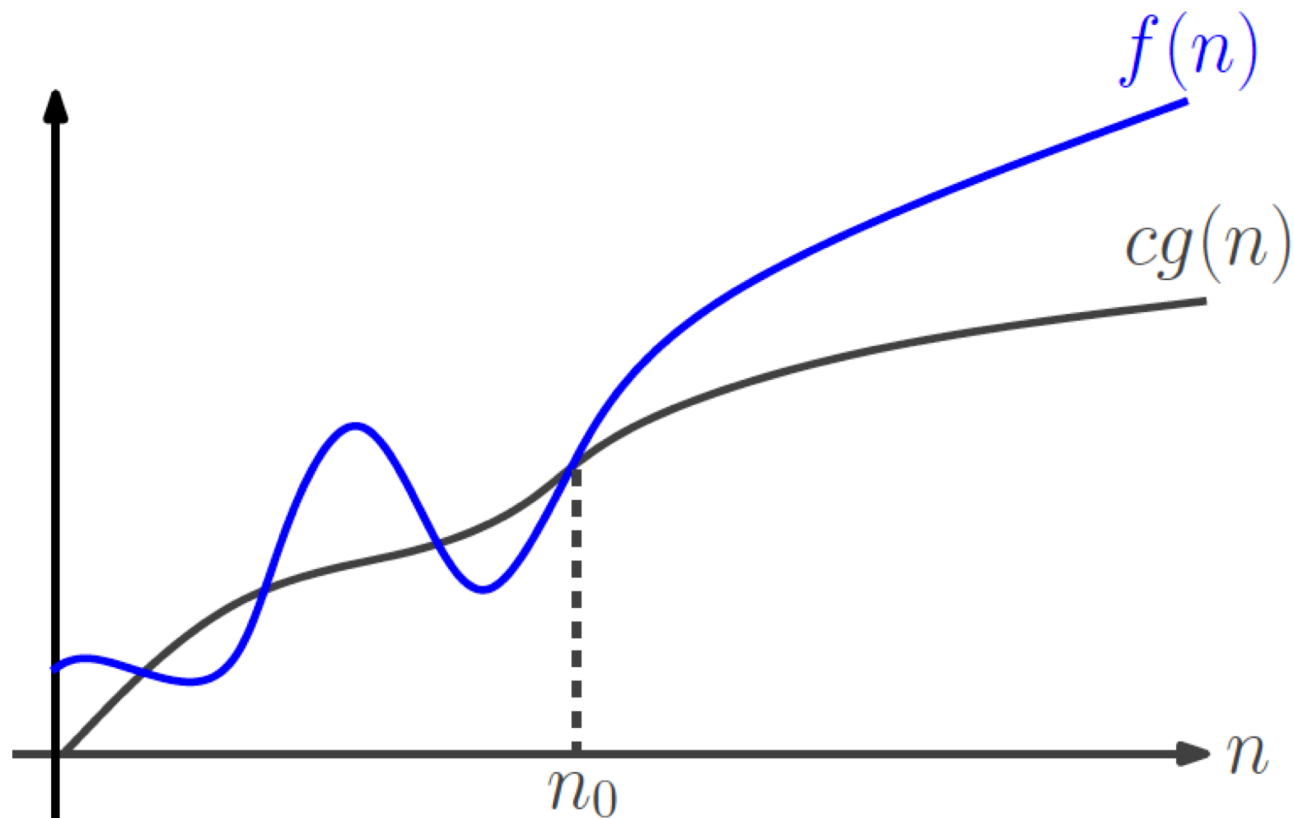
Asymptotic lower bound on a function

Ω Notation

Definition

$f(n) = \Omega(g(n))$ if and only if
 $\exists c > 0, \exists n_0 > 0 \ni \forall n \geq n_0, 0 < c.g(n) \leq f(n).$

Asymptotic lower bound on a function



Asymptotic tight bounds on a function

Θ -notation

We use Θ -notation for *asymptotically tight bounds*. For a given function $g(n)$, we denote by $\Theta(g(n))$ (“theta of g of n ”) the set of functions

$$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that} \\ 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0\} .$$

Asymptotic tight bounds on a function

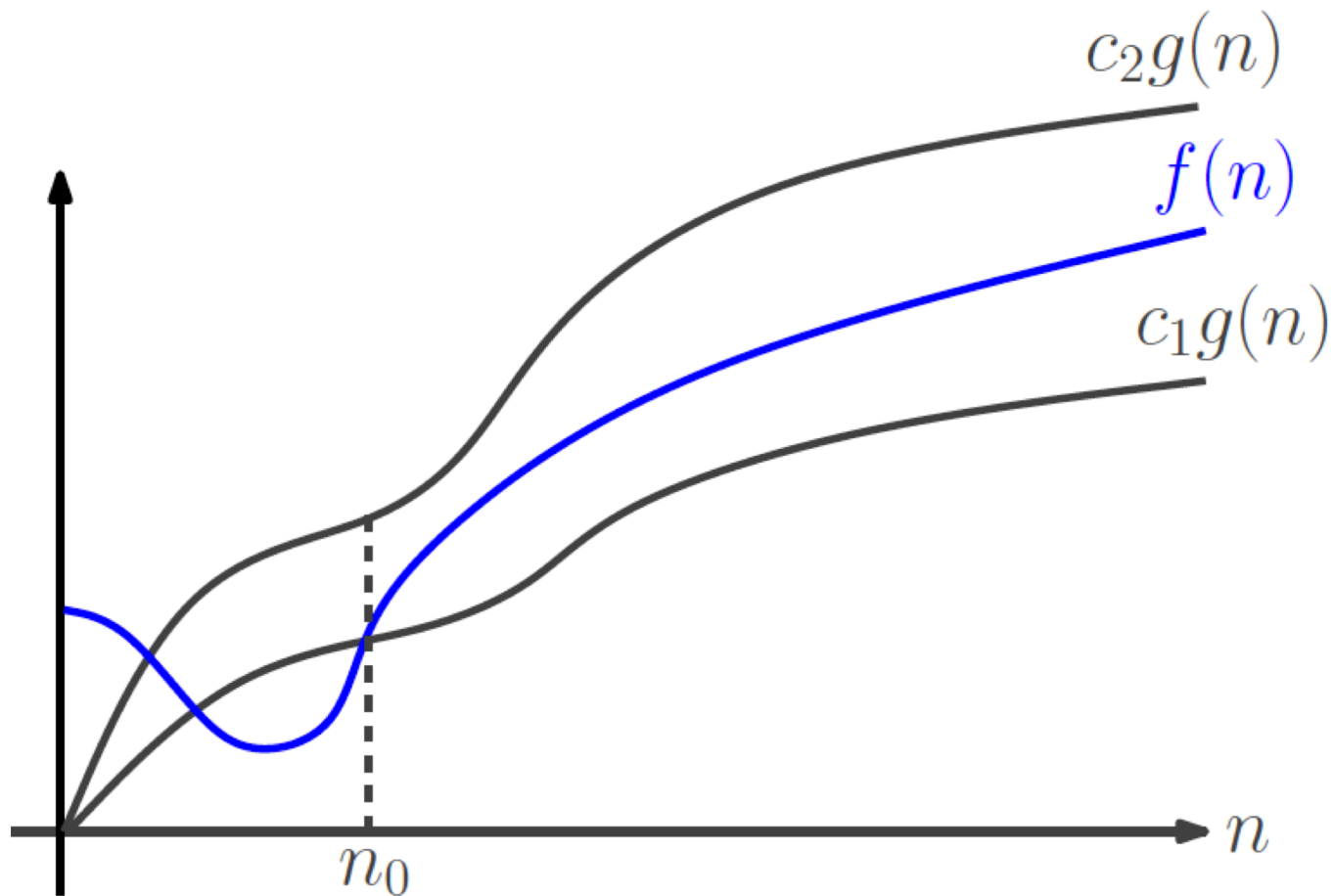
Θ Notation

Definition

$f(n) = \Theta(g(n))$ if and only if

$$\exists c_1, c_2 > 0, \exists n_0 > 0 \ni \forall n \geq n_0, 0 < c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n).$$

Asymptotic tight bounds on a function

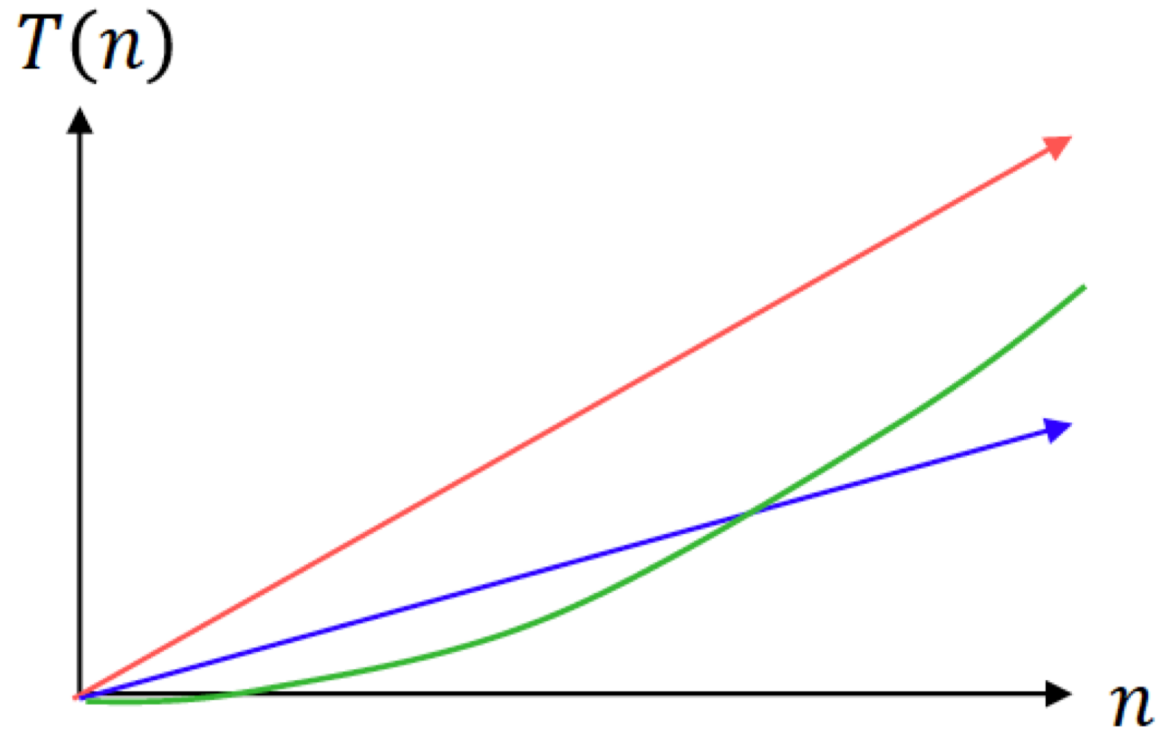


Function growth

$$\underline{f(n) = n}$$

$$\underline{g(n) = 4n}$$

$$\underline{h(n) = n^2}$$



Complexity Classes

Class	Big O	If you double N...	Example algorithm
constant	$O(1)$	unchanged	Add to front of linked list
logarithmic	$O(\log n)$	Increases slightly	Binary search
linear	$O(n)$	doubles	Sequential search
"n log n"*	$O(n \log n)$	Slightly more than doubles	Merge sort
quadratic	$O(n^2)$	quadruples	Nested loops traversing a 2D array
cubic	$O(n^3)$	Multiplies by 8	Triple nested loop
polynomial	$O(n^c)$		
exponential	$O(c^n)$	Increases drastically	

Complexity Classes

